

Rapport IA

Groupe 19 : Mister Jack



Sundman--Mathon Thomas

Medina Thomas

Iangaev Michel

Medini Sacha

Reynaud Nathan

Dumarché Dorian

Testelin Maxime

Dima Dalloul - Tutrice De Projet

Remerciements

Nous remercions notre tutrice Dima DALLOUL, les enseignants et encadrants du projet pour leurs conseils et leur accompagnement tout au long du développement.

Nous remercions également les personnes ayant participé aux tests et ayant permis d'améliorer le jeu et les différentes intelligences artificielles.

Enfin, ce projet nous a permis d'approfondir nos connaissances en Java, en algorithmique et en intelligence artificielle, notamment autour de l'algorithme MinMax et de ses optimisations.

Table des matières

Remerciements	2
Table des matières	3
Description du jeu	4
Idées de base	6
Estimation du facteur de branchement	6
Réflexion autour de l'heuristique	8
IA et heuristique	9
La fonction d'évaluation	9
Fonctionnement de l'évolution des poids	10
Evolution des poids sur 45 cycles réalisée après le rendu	11
Difficulté des IA	16
Tests statistiques	16
Conclusion	21
Annexe	22

Description du jeu

Mister Jack Pocket est un jeu asymétrique opposant deux joueurs : l'enquêteur et Jack.

L'objectif de l'enquêteur est d'identifier Jack parmi les 9 suspects, tandis que Jack doit récupérer 6 sabliers avant d'être découvert.

Le plateau est composé de 9 districts disposés en grille 3×3.

Chaque district contient : un mur et un personnage (qui peut être masqué une fois innocenté).

Trois détectives sont placés autour du plateau.

Le jeu comporte également 4 jetons d'action, chacun possédant deux faces :

- Jeton 1 : Alibi / Holmes
- Jeton 2 : Toby / Watson
- Jeton 3 : Rotation / Échange
- Jeton 4 : Rotation / Joker

Actions possibles

- Alibi : pioche une carte alibi pouvant innocenter un suspect ou faire gagner des sabliers à Jack.
- Toby / Watson / Holmes : déplace un détective de 1 ou 2 cases.
- Joker : déplace un détective d'une case ou ne réalise aucune action.
- Rotation : fait pivoter un district.
- Échange : échange la position de deux districts.

Déroulement d'un tour

Lors des tours impairs :

- les jetons sont placés aléatoirement sur une de leurs deux faces ;
- l'enquêteur joue un jeton ;
- Jack joue ensuite deux jetons ;
- l'enquêteur joue le dernier jeton.

Lors des tours pairs :

- les jetons sont simplement retournés ;
- Jack commence ;
- l'enquêteur joue ensuite deux actions ;
- Jack joue la dernière action.

Appel à témoin

À la fin de chaque tour, une phase d'appel à témoin a lieu.

Jack indique alors s'il est visible par un détective :

- Si Jack est visible, tous les suspects non visibles sont innocentés ;
- Sinon, tous les suspects visibles sont innocentés.

Si Jack n'est pas visible, il gagne également un sablier.

Quand les 2 joueurs atteignent leurs objectifs dans le même tour, on passe en mode course poursuite et les objectifs changent :

- pour l'enquêteur : que le Jack devienne visible par un détective en fin de tour
- pour le Jack : rester invisible jusqu'à la fin du tour 8

Exemple visuel :



Idées de base

Estimation du facteur de branchement

Le nombre de possibilités

Selon le jeton choisi il n'y a pas le même nombre de coups possibles :

- Jeton alibi : 1 seule possibilité
- Jeton détective (1 par détective) : 2 possibilités
- Jeton Joker : 3 possibilités ou 4 pour Jack car peut ne rien faire
- Jeton Rotation (2 dans la partie) : 27 ($9 * 3$) possibilités
- Jeton Échange : 72 ($9*8$) possibilités

Nombre de possibilités durant un seul tour :

- Pire cas : Jetons = (Homes, Toby/Watson, échange, Rotation)
ce qui donne un total de possibilités : $2*2*72*27 = \underline{7\,776}$
- Meilleur cas : Jetons = (Alibi, Toby/Watson, Rotation, Joker)
ce qui donne un total de possibilités : $1*2*27*4 = \underline{216}$
- Cas Moyen : 16 combinaisons de jetons mais le jeton 2 a toujours 2 possibilités donc que 8 combinaisons pour le calcul ($1*2*72*27 + 1*2*72*4 + 1*2*27*27 + 1*2*27*4 + 2*2*72*27 + 2*2*72*4 + 2*2*27*27 + 2*2*27*4$) / 8 = $\underline{2\,301.75}$

Nombre de possibilités pour 2 tours d'affilée (ia omnisciente) :

- Cas du tour impair, on retourne les jetons donc tour suivant prévisible ce qui donne un total de ($2*1*2*2*72*27*27*4$) = $\underline{1\,679\,616}$ possibilités à calculer
- Cas du tour pair, on lance les jetons donc plus imprévisible :
 - Pire cas ($7776 * 7776$) = $\underline{60\,466\,176}$
 - Meilleur cas ($216 * 216$) = $\underline{46\,656}$
 - Cas moyen ($2301.75 * 2301.75$) = $\underline{5\,298\,053.0625}$

Nombre de possibilités pour 2 tours d'affilée (ia non omnisciente) :

- Pour les tours impairs, le calcul reste le même étant donné qu'il est prévisible
- Pour les tours pairs, les calculs sont différents. Vu que l'on ignore la face les jetons au prochain tour, il est impératif de toutes les

simuler : (Moyenne d'un tour) * (nombre de tour possible) * tour actuel. Nous obtenons :

- Cas moyen : $2301.75 * 16 * 2301.75 = 84\,768\,849$
 - Meilleur cas : $2301.75 * 16 * 216 = 7\,954\,848$
 - Pire cas : $2301.75 * 16 * 7776 = 286\,374\,528$
- Il faut aussi prendre en compte que lors de la simulation de l'enquêteur, ce taux est doublé car étant donné qu'on ignore si jack sera visible lors de l'appel à témoins, il est nécessaire d'évaluer le cas où jack est visible et celui où jack est invisible.

Optimisation possible :

- Quand il ne reste que 3 jetons, un joueur joue 2 fois d'affilée. Or, effectuer l'action A puis B revient au même que B puis A. On peut donc éviter de recalculer ces cas symétriques et réduire d'environ moitié le nombre de combinaisons à explorer.
Exemple : au lieu de calculer (A,B) et (B,A), un seul des deux cas est conservé (optimisation réalisée avec des *skips* dans le MinMax).
- Ajout des alpha bêta dans le MinMax, quoique difficile à estimer, permet sur des profondeurs similaires de passer d'une vingtaine de secondes à 5-6 secondes.

Bilan

Nous constatons une différence marquée entre le pire cas et le meilleur. Ainsi, une situation donnée permet de simuler plus ou moins de coups à l'avance, comme dans l'exemple des deux tours d'affilée simulés par une IA non omnisciente où le coût de calcul d'un tour pair peut dépasser largement celui d'un tour impair, même dans les cas favorables.

Réflexions autour de l'heuristique

Étant donné le côté asymétrique du jeu, les 2 joueurs ne peuvent pas avoir le même calcul d'heuristique.

Les paramètres les plus évidents sont ceux liés à leurs conditions de victoire, c'est-à-dire pour Jack, le nombre de sabliers et pour l'enquêteur, le nombre de suspects éliminés.

Bien sûr, les paramètres d'un rôle peuvent être par conséquent utiles à l'autre, ce qui donnerait :

- Pour Jack : nombre de sabliers + nombre de suspects encore présents
- Pour l'enquêteur : $9 - \text{nombre de suspects encore présents} - \text{nombre de sabliers déjà piochés}$

Enfin, l'ajout d'un poids aléatoire dont l'ampleur dépend de la difficulté de l'IA afin d'empêcher un déterminisme de l'IA a été réalisé.

Recherche davantage poussée

Nous avons pu constater que certaines configurations sont plus avantageuses envers Jack ou envers l'enquêteur, selon la position des tuiles, celle des inspecteurs ou celle de Jack par exemple. Aussi, l'heuristique de base donnée précédemment s'applique à la partie en global mais pas au sein du même tour (4 actions). Or, uniquement l'action alibi peut affecter directement les sabliers et le nombre de suspects.

Il faut donc de nouveaux paramètres pour évaluer le tour en cours. Ainsi, pour l'enquêteur, nous avons ajouté notamment la qualité du split (essaye le plus possible de faire 50/50 entre suspect visible et invisible). Pour Jack, il s'agit du contraire. Son objectif principal durant un tour est de maximiser son anonymat et donc d'être dans le plus grand groupe (visible ou invisible) avec un bonus pour l'invisibilité car cela lui rapporte des sabliers.

En plus de cela, l'objectif change durant la phase de course poursuite (arrive plutôt vers la fin de partie) et donc il ne faut plus prendre en compte certains paramètres et utiliser principalement celui concernant la visibilité de Jack.

IA et heuristique finale

Même si les paramètres choisis auparavant sont assez bons, nous ne sommes ni certains de leur pertinence ni de leur poids respectifs dans l'heuristique. Nous avons donc décidé de faire des simulations faisant évoluer les poids en les améliorant génération après génération, pour trouver les meilleurs poids possibles.

La fonction d'évaluation finale

Pour chacun des rôles, nous avons choisi 6 paramètres. Chaque paramètre est calculé et normalisé avant d'être multiplié par des poids qui diffèrent selon le rôle et le moment de la partie : début (tour 1 à 3), milieu (tour 4 et 5) et fin (tour 6 à 8).

Les paramètres d'évaluation pour l'enquêteur :

- 1. Qualité dichotomique : À quel point le nombre de suspects visibles/invisibles fait un 50/50, pour être plus certain d'éliminer des suspects.
- 2. Répartition des inspecteurs : Le nombre de suspects vus des inspecteurs sans que plusieurs inspecteurs ne regardent les mêmes suspects
- 3. Proximité des suspects : La distance moyenne séparant un enquêteur d'un suspect, favorisant la proximité
- 4. Gain alibis : Le total d'alibis gagnés (utile lors d'évaluations portant plus loin que la fin du tour)
- 5. Nombre de suspects innocentés : Le nombre de suspects déjà éliminés (utile lors d'évaluations portant plus loin que la fin du tour)
- 6. Nombre de sabliers refusés : nombre de tours où Jack, étant visible lors de l'appel à témoins, n'a pas pu récupérer le sablier de fin de tour (utile lors d'évaluations portant plus loin que la fin du tour)

POIDS_ENQ_DEBUT = [0.01, 0.30, 0.09, 0.12, 0.22, 0.22]

POIDS_ENQ_MILIEU = [0.13, 0.22, 0.18, 0.28, 0.13, 0.03]

POIDS_ENQ_FIN = [0.17, 0.15, 0.16, 0.9, 0.09, 0.30]

Nous pouvons constater que tous les paramètres de la fonction heuristique correspondant à l'enquêteur semblent pertinents d'après les données recueillies, car le poids d'un même paramètre est supérieur à 0.1 (c'est-à-dire 10%) dans au moins un des trois tableaux (Début, Milieu, Fin).

Les paramètres d'évaluation pour Jack :

- 1. Visibilité : Visibilité de Jack lors de l'appel à témoins
- 2. Exposition visuelle : Depuis combien de positions enquêteurs Jack pourrait être visible
- 3. Protection : À quel point Jack est emmuré (moyenne de murs séparant Jack d'une position enquêteurs depuis laquelle il pourrait être visible)
- 4. Nombre de suspects restants : Si Jack visible, nombre de suspects visibles, autrement le nombre de suspects invisibles
- 5. Gain sabliers : Le total de sabliers gagnés (utile lors d'évaluations portant plus loin que la fin du tour)
- 6. Stabilité du plateau : À quel point une action portant sur une tuile peut affecter l'état de visibilité d'un grand nombre de suspects

POIDS_JACK_DEBUT = [0.03, 0.03, 0.0, 0.41, 0.51, 0.01]

POIDS_JACK_MILIEU = [0.26, 0.0, 0.08, 0.36, 0.27, 0.01]

POIDS_JACK_FIN = [0.15, 0.13, 0.33, 0.11, 0.23, 0.05]

Ces résultats, de même que ceux concernant l'enquêteur, mettent en évidence que tous les paramètres de Jack semblent pertinents, à part le dernier, c'est-à-dire la stabilité du plateau, dont le poids dans les trois tableaux est très restreint et varie entre 0.01 et 0.05, c'est-à-dire entre 1 et 5%.

Fonctionnement de l'évolution des poids

Nous avons créé des profils génétiques correspondant à l'enquêteur ou à Jack et contenant trois tableaux de 6 poids (Début, Milieu, Fin). Nous avons choisi de faire évoluer Jack et l'enquêteur périodiquement, où le cycle suivant est répété :

évolution de Jack:

- récupération des 100 profils Jack et des 100 profils inspecteur sauvegardés (ces profils sont créés aléatoirement au premier cycle)
- affrontement des profils Jack contre les profils enquêteur, récupération des profils Jack les plus performants, croisements en mutations de ces profils pour obtenir la génération suivante de 100 Jack
- à la trentième génération, sauvegarde des 100 profils Jack

suivie de l'évolution de l'inspecteur:

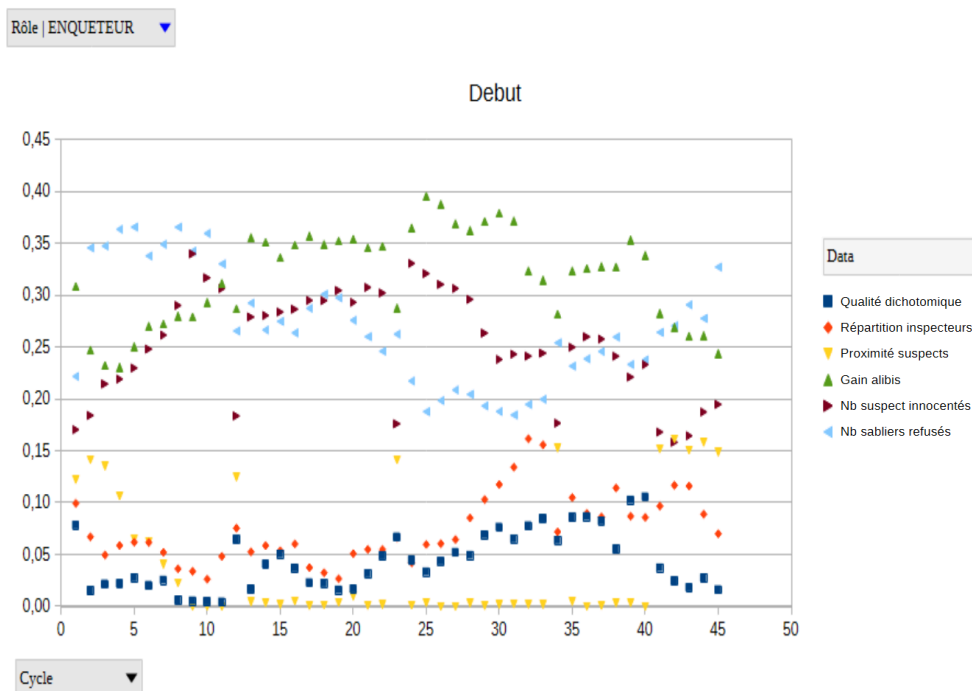
- récupération des 100 profils inspecteur et des 100 profils Jack sauvegardés (ces profils sont créés aléatoirement au premier cycle)
- affrontement des profils inspecteur contre les profils Jack, récupération des profils inspecteur les plus performants, croisements en mutations de ces profils pour obtenir la génération suivante de 100 inspecteurs
- à la trentième génération, sauvegarde des 100 profils inspecteur

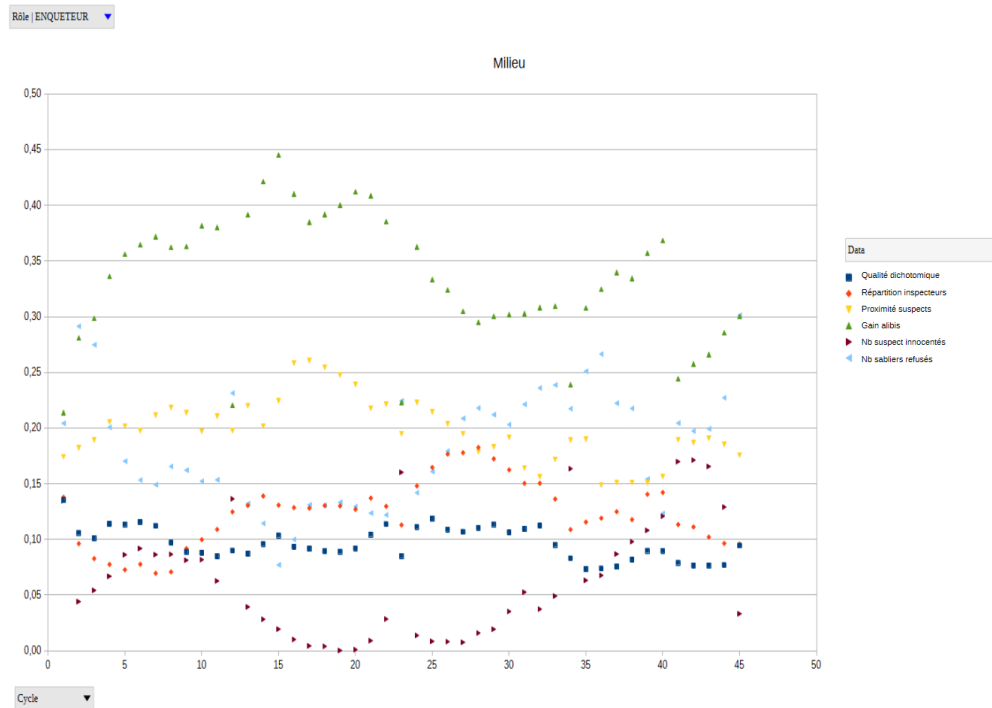
Evolution des poids sur 45 cycles réalisée après le rendu

Voici respectivement l'évolution des tableaux de poids de l'enquêteur et de Jack. Les poids sont des réels entre 0.0 et 1.0 (et au sein de chaque tableau, la somme des poids vaut 1.

Pour l'enquêteur :

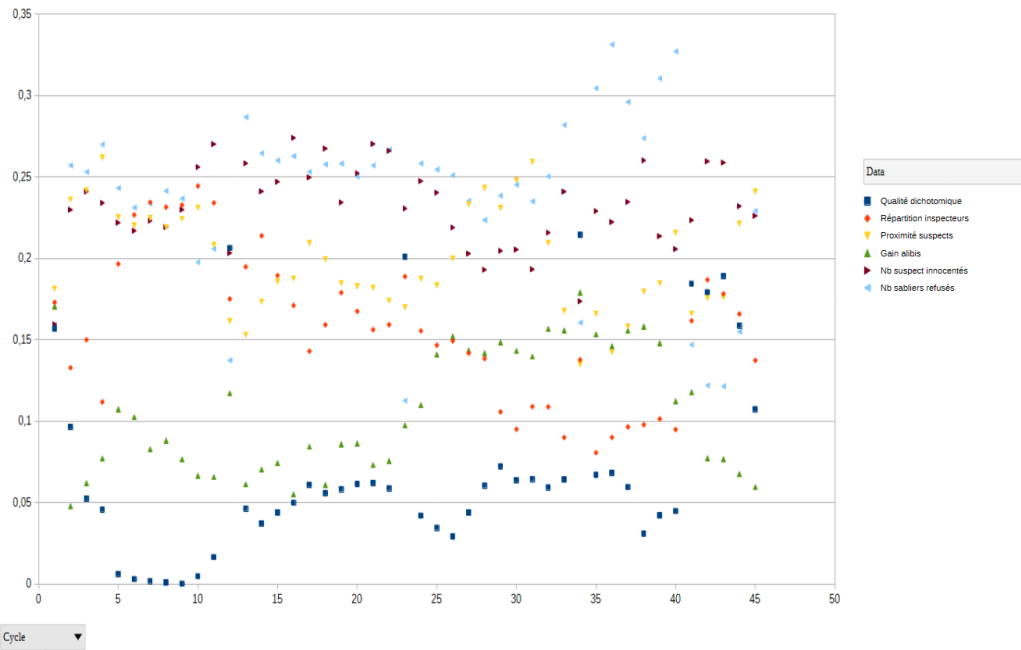
Début :



Milieu :*Fin :*

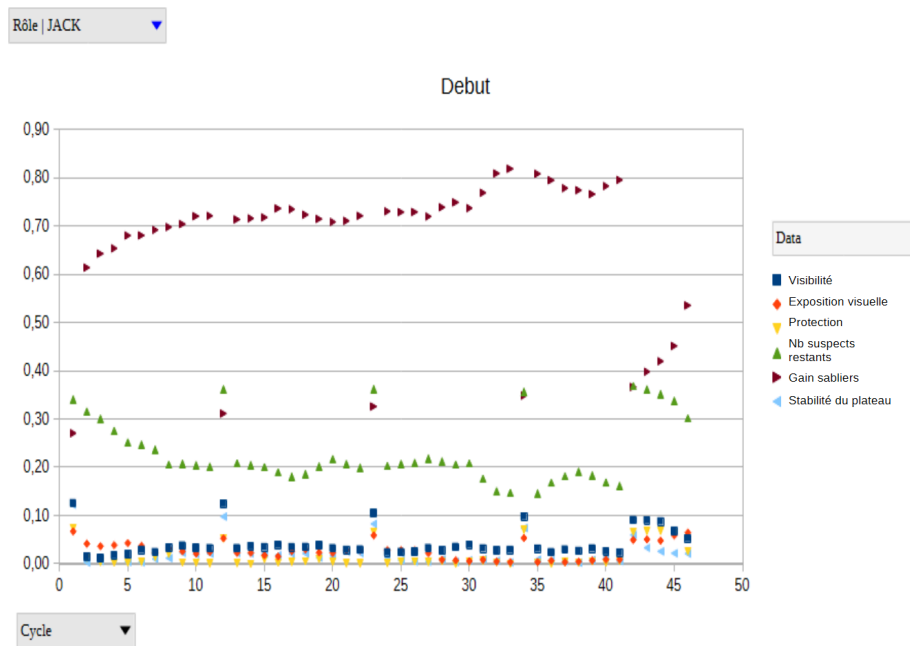
Rôle | ENQUÊTEUR ▼

Fin

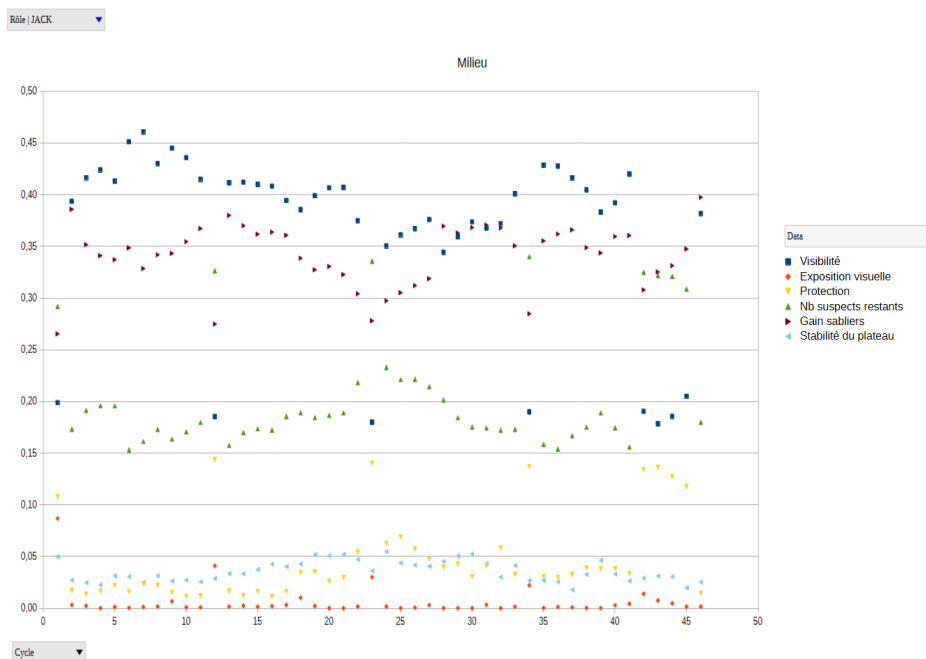


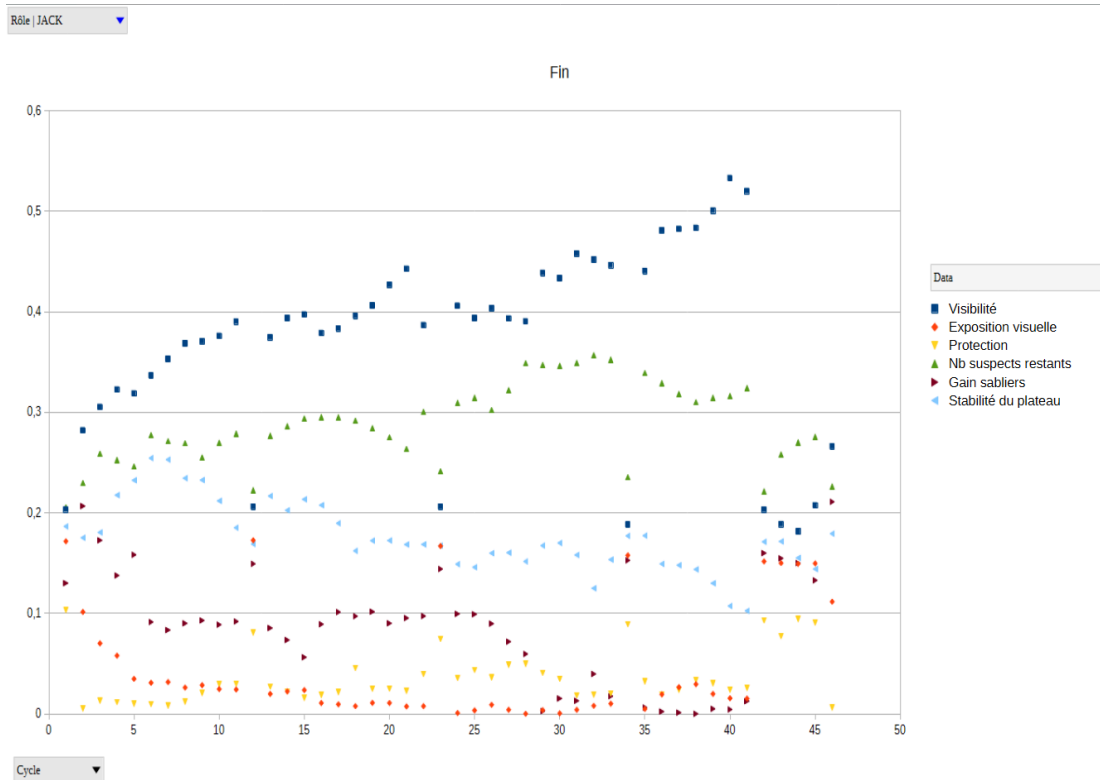
Pour Jack :

Début:



Milieu:





Nous constatons une évolution stable pour l'ensemble des paramètres ainsi que des écarts entre ces derniers qui paraissent logique comme par exemple, des poids plus importants sur les paramètres portant sur le nombre de suspects restants ainsi que le gain de sabliers. Aussi, nous observons des écarts au sein d'un même paramètre, entre les trois tableaux, montrant une fluctuation de l'importance d'un paramètre durant la partie. Cela laisse penser que l'évolution s'est déroulée correctement et que les paramètres Qualité dichotomique pour l'enquêteur et Exposition visuelle et Protection pour Jack semblent moins pertinents que les autres car se stabilisent en dessous de 0.1. La notoriété des paramètres accumulatifs peut être expliquée par l'utilisation de l'heuristique uniquement sur les feuilles du min max et non au niveau des nœuds.

Difficulté des IA

Toutes les IA utilisent le MinMax avec des profondeurs différentes et avec un taux d'aléatoire plus ou moins important pour régler la difficulté.

- L'IA Facile : Profondeur 1 pour l'enquêteur et 2 pour le jack et taux d'aléatoire à 0.5.
- L'IA Moyenne : profondeur 2 pour l'enquêteur et 3 pour le jack et taux d'aléatoire à 0.3.
- L'IA Difficile : profondeur variable selon la situation allant de 3 pour les pires cas et 6/7 pour les meilleurs cas, et un taux d'aléatoire à 0.01.
- L'IA Experte : non réalisée mais on peut l'imaginer avec une profondeur un peu plus grande que la difficile en s'autorisant plus de temps de calcul.

Tests et statistiques

Test du taux de victoire de Jack selon les difficultés sur 1000 parties					
	Enq Facile	Enq Normal	Enq Difficile	Enq Random	En Moyenne
Jack Facile	31.9%	12.2%	10.5%	68.3%	30.72%
Jack Normal	75.7%	50.4%	31.4%	94.9%	63.1%
Jack Difficile	87.7%	74.2%	49.5%	96.9%	77.08%
Jack Random	7.5%	1.5%	2%	28.9%	9.97%
En Moyenne	50.7%	34.57%	23.35%	72.25%	45.22%

- On peut voir avec ces stats que les IA en difficile battent en grande majorité les normal et facile et que les normal battent les faciles
- L'objectif était également d'obtenir un équilibre proche de 50/50 lorsque deux IAs de même niveau s'affrontent. Cet équilibre est relativement bien atteint pour les difficultés normales et difficiles, mais moins pour la difficulté facile. Cela s'explique notamment à cause de l'avantage naturel pour le rôle d'enquêteur.

Test des temps de calcul selon les difficultés sur 1000 parties				
	Enq Normal	Enq Difficile	Jack Normal	Jack Difficile
Moyenne	0 ms	31 ms	5 ms	89 ms
Moyenne de partie Max	2 ms	194 ms	34 ms	402 ms
Max partie en moyenne	0 ms	157 ms	29 ms	398 ms
Maximum	9 ms	1514 ms	271 ms	1423 ms

Nous pouvons voir que le temps maximum dans le difficile est vers 1,5 s donc ne dépasse pas l'objectif qu'on s'était donné de 4 secondes max.

Nous aurions pu optimiser plus le temps moyen sur l'IA difficile pour faire plus de calcul, mais cela ne nous a pas semblé pertinent car après plusieurs tests humains (et statistiques du tableau précédent), la difficulté semblait déjà ajustée.

Test des poids la fonction heuristique (taux de victoire de Jack)			
	Jack Poids à 1	Jack Statique	Jack Changeant
Enq Random	453 / 500	491 / 500	483 / 500
Enq poids à 1			139 / 300
Enq Statique		128 / 300	157 / 300
Enq Changeant	103 / 300	126 / 300	162 / 300

Petite conclusion sur la base de ce tableau :

- Poids à 1 veut dire que tous les paramètres ont le même poids
- Statique veut dire que les poids sont différents selon les paramètres, mais ces poids ne changent pas au cours de la partie

- Changeant les poids change si on est au début, milieu ou vers fin de partie
- Random se contente de jouer des coups randoms sans utiliser d'heuristique ou de MinMax

Pour le Jack, on a décidé de prendre celui qui est changeant selon la situation de la partie car le plus performant. Cependant, pour l'enquêteur, la différence entre statique et changeant n'est pas aussi évidente, on a finalement décidé de garder le changeant.

On a fait d'autres tests où on avait réussi à rendre l'enquêteur encore plus performant, mais nous avons choisi de ne pas utiliser ces changements car l'enquêteur devenait trop fort et brisait le côté proche du 50/50 et devenait trop dur pour un humain même pour le difficile.

On a continué d'autres tests après le rendu pour essayer d'avoir les poids "parfaits" pour l'enquêteur et Jack.

Les tests suivants sont les résultats contre des joueurs humains de différents niveaux de jeu, mais la majorité étaient des joueurs expérimentés.

Test Humain contre IA (taux de victoire humain)			
	IA Facile	IA Moyenne	IA Difficile
Humain Jack	9/10	6/10	4/10
Humain Enq	10/10	6/10	4/10

Les résultats de nos tests sont conformes à notre volonté de difficulté.

Les joueurs expérimentés gagnent facilement contre l'IA facile, gagnent en majorité contre la moyenne et la difficile donne un plus gros défi où la plupart des parties vont souvent jusqu'à la phase de "course poursuite" donc vers une fin de partie assez serrée.

Toutes les IA (même la Difficile) font parfois des erreurs qu'un humain n'aurait sûrement pas faites, mais cela permet d'équilibrer et rendre la victoire possible pour un humain, laissant une possibilité pour une IA Experte.

Stats Bonus faits après le rendu du code pour avoir les meilleures heuristiques pour une potentielle IA experte :

Test nombre de victoire Jack selon cycle (En difficile)					
	Enq Rendu	Enq 45 c	Enq 35 c	Enq 25 c	Enq 15 c
Jack Rendu		245/500	241/500	98/200	97/200
Jack 45 c	233/500	96/200	92/200	83/200	87/200
Jack 35 c	238/500	91/200	81/200	83/200	99/200
Jack 25 c	66/200	73/200	70/200	78/200	92/100
Jack 15 c	76/200	90/200	99/200	107/200	89/200

Test nombre de victoire Jack selon cycle (En Moyen)					
	Enq Rendu	Enq 45 c	Enq 35 c	Enq 25 c	Total
Jack Rendu	240/500	291/500	297/500	308/500	1136/2000
Jack 45 c	267/500	323/500	292/500	333/500	1215/2000
Jack 35 c	295/500	330/500	298/500	358/500	1281/2000
Jack 25 c	257/500	303/500	289/500	331/500	1180/2000
Total	1059/2000	1247/2000	1176/2000	1330/2000	4812/8000

Malheureusement, les calculs faits ne sont pas encore assez concluants pour pouvoir dire que nous avons trouvé une heuristique pour IA Experte (le plus imbattable possible).

Test nombre de victoire Jack selon cycle (Difficile)		
	Enq Rendu	Enq 11
Jack Rendu	263/500	275/500
Jack 11	239/500	235/500

Test nombre de victoire Jack selon cycle (Moyen)		
	Enq Rendu	Enq 11
Jack Rendu	240/500	345/500
Jack 11	276/500	360/500

Conclusion

Ce projet nous a permis de développer une intelligence artificielle pour le jeu *Mister Jack Pocket* à l'aide de l'algorithme MinMax et de différentes optimisations comme l'alpha-bêta.

Nous avons également travaillé sur la conception d'heuristiques adaptées au caractère asymétrique du jeu, ainsi que sur l'équilibrage des différentes difficultés d'IA grâce à de nombreux tests statistiques et humains.

Les résultats obtenus sont globalement satisfaisants : les différentes IA proposent des comportements cohérents avec leur niveau de difficulté et les temps de calcul restent raisonnables même dans les cas les plus complexes.

Enfin, plusieurs pistes d'amélioration restent possibles, notamment l'optimisation des heuristiques et le développement d'une IA experte plus performante.

Annexe

- URL du Git

<https://gricad-gitlab.univ-grenoble-alpes.fr/rendus-prog6-2025-2026/groupe-19>

- Page du jeu

<https://boardgamegeek.com/boardgame/72287/mr-jack-pocket>